Matter Bridgeを介した非Matter機器の操作におけるControllerの検証可能範囲と限界の分析

吉永 達哉[1,a)]　依田 みなみ[1,b)]　松野 裕[1,c)]

概要：スマートホーム規格 Matter は，IoT 機器を操作・管理する Controller と，非 Matter 機器 (下位機器) を Matter ネットワークへ接続する Bridge について定める．Controller は下位機器と直接通信せず，Bridge の Endpoint を通じて操作や状態を取得する．しかし，Bridge が報告する情報と実際の下位機器との対応を Controller が検証する手段は，仕様上規定されていないため，Bridge による偽装を検知できずに攻撃の起点となるおそれがある．そこで本研究では，Matter 1.6 の Bridge 規定の分析，および公式 Matter SDK の Bridge サンプルと市販 Bridge 製品を用いた実機検証の二段階により，Bridge 内部の不整合が Controller の取得情報に現れるかを検証した．実機検証では二つの結果を得た．第一に，SDK サンプル実装では，下位機器との対応関係，状態の反映経路，およびコマンドの転送先を変更しても，Bridge 経由の取得情報に差は現れなかった．第二に，市販 Bridge 製品では，スマートロックを照明として登録でき，偽装した消灯操作によって実機が施錠された．以上より，Bridge が返す情報のみでは状態の正しさを検証ができず，下位機器の直接観測や Bridge 内部処理の完全性確認といった追加の手段が必要であることが示唆された．

キーワード：Matter，Matter Bridge，スマートホーム，IoT セキュリティ

Analysis of the Scope and Limitations of Controller-Side Verification for Non-Matter Device Operations via Matter Bridges

Tatsuya Yoshinaga[1,a)]　Minami Yoda[1,b)]　Yutaka Matsuno[1,c)]

Abstract: The smart home standard Matter specifies Controllers, which operate and manage IoT devices, and Bridges, which connect non-Matter devices, hereafter called downstream devices, to a Matter network. A Controller does not communicate directly with downstream devices; it operates them and obtains their state through Endpoints exposed by the Bridge. However, the specification defines no means for a Controller to verify the correspondence between the information a Bridge reports and the actual downstream devices, so spoofing by a Bridge may go undetected and serve as a starting point for attacks. We therefore examined whether inconsistencies inside a Bridge appear in the information a Controller obtains, in two stages: an analysis of the Matter 1.6 provisions for Bridges, and experiments on the official Matter SDK Bridge sample and a commercial Bridge product. The experiments yielded two results. First, in the SDK sample implementation, altering the mapping to downstream devices, the state-reflection path, or the command destination produced no difference in the information obtained through the Bridge. Second, in the commercial product, a smart lock could be registered as a light, and a spoofed Off operation locked the physical device. These results indicate that the information a Bridge returns is insufficient on its own to verify the correctness of the reported state, and that additional means are required, such as direct observation of downstream devices or confirmation of the integrity of the Bridge's internal processing.

Keywords: Matter, Matter Bridge, smart home, IoT security

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

1. はじめに

スマートホームの IoT 機器（機器）は製造元ごとに通信方式や操作方法が異なるため，複数製品を一元的に管理することが困難であった．この課題に対し，Amazon，Apple，Google，Samsung SmartThings などが参加する Connectivity Standards Alliance(CSA) は，機器間の相互運用を目的とした共通規格 Matter を策定した [1], [2], [3], [4]．Matter は機器種別ごとに機能，操作方式，状態出力を共通化し，異なる製造元の機器を同一の手順で扱えるようにする規格である．

Matter 非対応の機器（下位機器）は，Bridge がを介して Matter ネットワークに接続する [1], [3]．この構成では，Matter の制御機器である Controller は下位機器と直接通信せず，Bridge の Endpoint を介して操作や状態取得を行う．したがって，Bridge が報告する情報等が実際の下位機器と一致していなければ，Controller は誤った状態判断や意図しない操作をするおそれがある．

Matter の安全性に関する先行研究は，機器と Controller 間の信頼確認の非対称性や，Controller および Fabric に関する利用者向け表示の不備を指摘してきた [5], [6]．Bridge についても，仕様上のリスク分析や異種規格との相互運用実装が報告されている [7], [8]．しかし，Bridge が報告する情報と下位機器との対応を，Controller がどの範囲まで検証できるかは明らかにされていない．

そこで本研究では, 以下の二つの Research Question(RQ) を設定し，Matter 1.6 の仕様分析，公式 Matter SDK の Bridge サンプル実装を用いた評価，および市販 Bridge 製品を用いた評価を行う．

RQ1(仕様上の検証機構): Matter 1.6 は，Bridge が報告する情報と下位機器との対応を Controller が検証する手段を，どのように規定しているか．

RQ2（不整合の検知可能性） Bridge 内部に不整合が生じたとき，その不整合は Controller が取得する情報にどのように現れるか．

本研究の貢献は二点である．第一に，Matter 1.6 の仕様分析により，Bridge が報告する情報と下位機器との対応を確認する共通の仕組みが規定されていないことを示した．第二に，公式 SDK サンプルと市販 Bridge 製品を用いて，Bridge 内部の不整合を Controller が取得情報から判別できない場合があることを実証した．SDK サンプルでは，下位機器との対応関係，状態の反映経路，およびコマンドの転送先を変更しても，Controller の取得情報に差は現れなかった．市販製品では，スマートロックを照明として登録でき，アプリの消灯操作で実機が施錠位置へ動いた．これらの結果は，Bridge が提示する情報の真偽を確定するには，報告された値そのものではなく，その値を生成する処理が正しく実行されたことを示す証拠が必要であることを示唆する．

[1] 日本大学
Nihon University
[a)] cstt26007@g.nihon-u.ac.jp
[b)] yoda.minami@nihon-u.ac.jp
[c)] matsuno.yutaka@nihon-u.ac.jp

2. Matter の機構と脅威モデル

2.1 Matter の構成要素と Bridge

図 1 に，Bridge を含む Matter の構成を示す．Matter に参加する個々の機器を Node と呼び，共通の認証局を信頼の基点として共有する Node の集合を Fabric と呼ぶ．Node が Fabric へ参加する手続を Commissioning と呼び，この過程では Device Attestation により機器の Matter 適合性と製造元が検証され，Node 単位で Node Operational Certificate（NOC）が付与される．Controller は，この NOC に基づく認証済みセッション上で，Interaction Model が定める Read，Subscribe，Invoke により他の Node を操作する [1]．

Node は，個別に識別可能な機能単位である Endpoint で構成される．各 Endpoint には機能を定義する Cluster が配置され，その内部に状態を表す Attribute と操作を表す Command が定義される．Device Type は Endpoint の役割と必要な Cluster の組み合わせを規定する [1], [3]．

Bridge は，Commissioning を経て Fabric に参加し，下位機器を Bridged Device として一つ以上の Endpoint で表現する．Controller は，Aggregator Endpoint の `PartsList` から配下の Endpoint 群をたどり，Bridged Node Device Type を持つ Endpoint を手がかりに各 Bridged Device を発見する [1], [3]．各 Bridged Device の最上位 Endpoint に置かれる Bridged Device Basic Information（BDBI）Cluster が識別子や到達性を，機能を表す Endpoint の On/Off Cluster 等が動作状態を保持・制御する．Bridge は，Matter Command を下位機器固有の制御コマンドへ変換・転送し，下位機器から得た状態変化を Attribute や Event へ反映する [1]．

2.2 脅威モデル

本研究では，Controller や通信経路の侵害，認証情報の漏えい，ならびに暗号学的破綻は仮定せず，基盤となるセキュア通信やアクセス制御は正常に機能しているものとする．Bridge Node は有効な NOC で Commissioning を完了しており，本研究は認証の迂回を不整合の原因としない．

この健全な通信基盤を前提とした上で，本研究は Bridge のアプリケーション層または下位プロトコル変換層における故障，あるいは意図的な改変を仮定する．故障時，または攻撃者により，Bridge が下位機器をどう扱うか，および Matter 上へ何を申告するかが変更され得る．ただし，Bridge が返すメッセージは構文，データ型，Interaction

© 2026 Information Processing Society of Japan

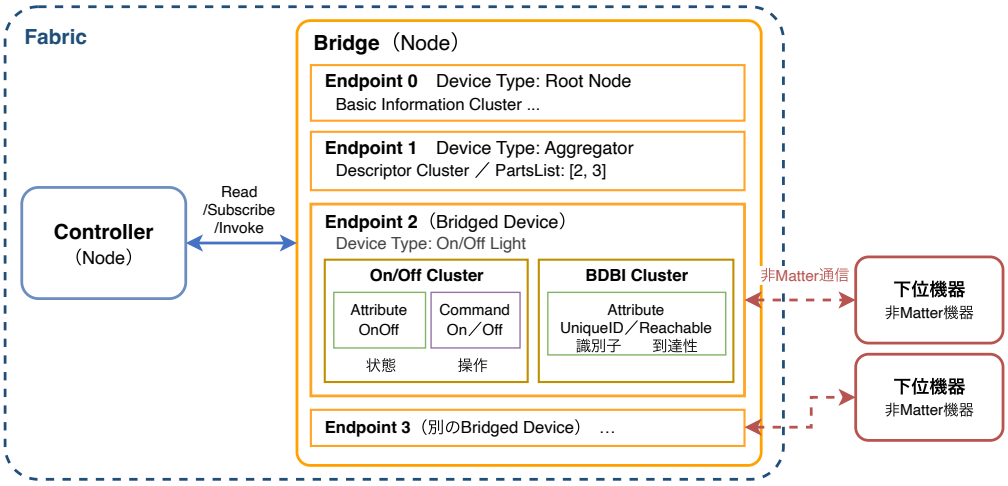


図 1　Bridge を含む Matter の構成要素

Fig. 1　Structure of a Matter network including a Bridge.

Model の応答形式を満たすものとし，形式違反だけで検知できるケースは扱わない．

この想定の下では，Bridge が処理する情報や制御と，下位機器との間に不整合が生じ得る．施錠や給湯のように物理的作用を伴う機器ではこうした不整合が安全に直結し，誤った状態申告は自動制御を通じて他の機器へも波及する．

2.3　Controller の観測範囲

Controller が，Bridge 内部で生じる不整合をどこまで検知できるかは，Controller 自身が取得できる情報の範囲に依存する．本研究では観測機器を，必要な権限を持つ単一の Controller Node とし，ベンダ固有の拡張機能には依存しないものとする．

Controller が用いる観測データは，認証済みセッション上で Read，Subscribe，Invoke を通じて取得した Endpoint 構成，Attribute，Event，Invoke 応答に限る．Controller は，ベンダ専用アプリ，クラウド API，デバッグ機能，Bridge 内部ログ，下位プロトコルのパケット傍受，追加の物理センサ，および他 Controller との相互比較といった Matter 外の手段を用いない．

この観測範囲は，本研究の結論が成り立つ条件を定める．以降で述べる検証の可否は，追加の情報源を持たない Controller が判断できる範囲のものである．

3.　RQ1：Controller による Bridge 処理の検証可能性

3.1　調査方法と分析対象

本研究は，Core §9.12 が規定する非 Matter 下位機器の Bridge 処理を対象とし，特定の製品に依存しない検証可能性を調べる．全 Bridge に共通する機構の有無は，Core §9.12 ならびに Root Node，Aggregator および Bridged Node に共通する必須要素から判定し，On/Off Light 固有の要素は P_S と P_C を具体化する対象として用いた [1], [3]．分析では，対象 Device Type の必須要素を主軸とし，条件付き要素および任意要素も補助候補として区別して調べた．

仕様要素は，Root Node，Aggregator，Bridged Node および実験対象の On/Off Light について，Base Device Type，Cluster Requirements および Element Requirements を順に適用して抽出した．Controller へ結果を提示し得る Descriptor Cluster（`PartsList` 等），BDBI（`UniqueID`，`Reachable` 等），On/Off Cluster（`OnOff`，`Off`，`On`，`Toggle`），Cluster instance の `DataVersion`，および Interaction Model の応答を候補とした [1], [2], [3]．規範根拠，候補の採否および範囲外とした機能は付録 A.1 に示す．

Core §9.12.2 の機器構成の公開，Core §9.12.4.2 の状態の反映，および Core §9.12.4.1 のコマンドの変換・転送に基づき，下位機器との対応に関わる Bridge 処理を次の三つの分析軸（P_I, P_S, P_C）として整理する．各分析軸と Controller の観測範囲との関係を**図 2** に示す．

- 機器構成の公開と対応付け P_I：Bridge が公開する Device Type，Endpoint および識別子が，下位機器と対応すること．
- 状態の取得と反映 P_S：Bridge が下位機器から取得した状態を，対応する Endpoint の Attribute や Event へ反映すること．
- コマンドの変換と転送 P_C：Bridge が Matter コマンドを下位機器向けの操作へ変換し，対応する下位機器へ転送すること．

3.2　仕様規定と Controller 観測の対応分析

機器構成の公開と対応付け P_I　Endpoint と下位機器との対応は，Matter の認証機構や Controller が観測できる構成情報だけでは独立に裏付けられない．

認証機構の観点では，Device Attestation は Commis-

© 2026 Information Processing Society of Japan

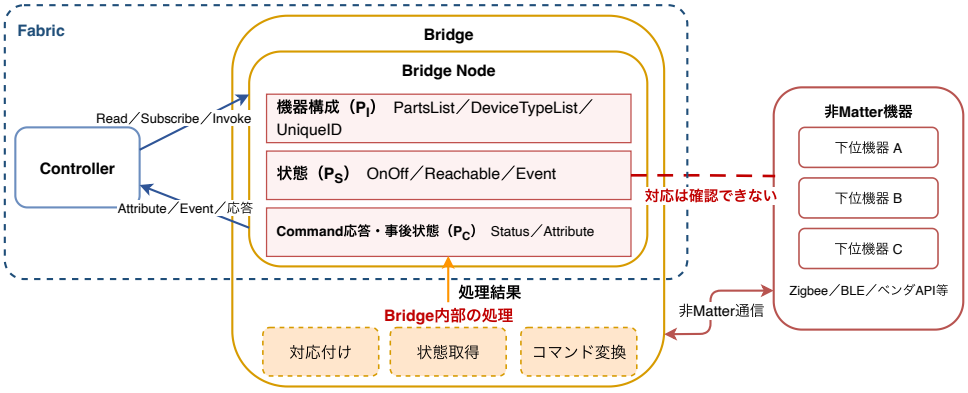


図 2 Bridge 内部の三つの処理，検証対象 P_I，P_S，P_C，および Controller の観測範囲
Fig. 2 Three verification targets and the Controller observation boundary.

sioning 対象である Bridge Node 自体を検証するにとどまる（Core §6.2.3）．Commissioning 時に発行される Node ID や NOC は Bridge Node に対して単一であり，個々の Bridged Device には割り当てられない（Core §9.12.8）．また，非 Matter 下位機器は Product ID を持たず，正規製品情報を管理する DCL（Distributed Compliance Ledger）にも登録されないため（Core §9.12.10），Bridge Node の認証成功は各 Endpoint と下位機器との同一性を保証しない．

次に構成情報の観点では，Controller は `PartsList`，`DeviceTypeList`，`ServerList` 等から構成構造を検査し，必須項目である `UniqueID` を追跡できる．しかし，下位機器自身が固有識別子を持たない場合，`UniqueID` の生成は Bridge に委ねられている（Core §9.13.5.3）．また，`VendorName` は Bridge が提示する値であり，製造元との一致を判定するには Bridge 外の事前情報を要する．`ConfigurationVersion` 等の構成管理属性が存在しても，下位機器のどの変化を構成変更と判定して更新するかは Bridge の実装依存である（Core §9.13.5.4）[1]．

さらに内部状態の維持に関して，仕様は再起動後も割り当て済みの Endpoint 番号を維持するよう要求する（Core §9.2.10）．しかし，これは番号の固定化を促す規定に過ぎず，その番号が指し示す対象が実機の同一性を保ち続けているかを Bridge とは独立に証明するものではない [1]．

したがって，下位機器と正しく対応した構成と，下位機器から乖離した不整合な構成であっても，構造制約を満たす限り Controller が得る観測値は同一となり得る．

状態の取得反映 P_S Bridge が報告する下位機器の状態情報は，その取得元や取得時点の正当性を Controller が直接検証することはできない．Core §9.12.4.2 は，下位機器から受信した value update，Event および Command を Matter 側の Attribute，Event および Command として表現するよう Bridge に要求する．本分析では，このうち状態を表す Attribute および Event を対象とする [1]．

Attribute の観点では，Controller が取得する `OnOff` や `Reachable` からは，その値が対応する下位機器から取得されたことや，下位機器で状態が取得された時点を確認できない．`Reachable`（Core §9.13.5.2）は Bridge による到達性の判定結果であり，仕様は偽陽性，偽陰性および検出遅延が生じ得ることを明記している．また，`DataVersion`（Core §7.10.3）は Cluster instance ごとのメタデータであり，Attribute が変化すると増加するが，初回公開時にランダムに初期化され，最大値を超えると 0 へ巡回する．したがって，Attribute の変化は把握できても，変更時刻や経過時間は確認できない．

次にイベントや時間情報の観点では，Event 時刻（Core §7.14.1，§7.14.1.2）は Matter Node 上で Event record が生成された時刻に過ぎず，下位機器での発生時刻を保証しない．なお，測定期間やタイムスタンプを保持する Cluster も存在するが，これらは特定 Cluster 固有の規定にとどまる上，その値自体も Bridge が提示するものに過ぎない [1], [2]．

したがって，下位機器の状態が変化していない場合と，下位機器の状態は変化したが Bridge がその変化を反映しない場合とで，Controller が得る状態情報は同一となり得る．以上より，Controller は報告された値のみから P_S における情報取得元や取得時点の正当性を検証することはできない．

コマンドの変換・転送 P_C Bridge に送信したコマンドが下位機器へ転送されたかを，Controller が応答や事後状態から直接検証することはできない．Core §9.12.4.1 は，Bridged Device 宛ての Matter コマンドを下位プロトコル向けに変換し，対象機器へ転送することを Bridge に要求する．本分析では，Controller による Invoke 応答と，実行後の Read で得る事後状態を観測対象とする [1], [2]．

Invoke 応答の観点では，本評価で扱う `Off`，`On`，`Toggle` の Response は「Y」と規定されており（Application §1.5.7），これは Status のみを返すことを意味する（Core §7.11）．この Status は InvokeResponseIB（Core §8.9.4.4）内の

© 2026 Information Processing Society of Japan

CommandStatusIB（Core §8.9.5.1）として表現されるが，その構成要素には，下位プロトコル上での実機への送信の有無や転送先を示す情報は含まれない．

次に事後状態の観点では，Command 送信後に Read で取得する Attribute 値だけからは，その値が下位機器の状態を反映したものか，Bridge 内部の状態のみを反映したものかを Controller 側で区別できない．

したがって，Command を意図した下位機器へ正常に転送した場合と，別の下位機器へ転送しつつ Bridge が同じ Invoke 応答と事後状態を返した場合とで，Controller が得る観測値は同一となり得る．以上より，Controller は Command が下位機器へ正しく転送されたことを Bridge 由来の情報のみから検証することはできない．

3.3 RQ1：仕様上の検証機構への回答

Matter 1.6 は Bridge に P_I，P_S，P_C に対応する処理を要求しているが，Controller がこれらの処理と下位機器との対応を確認するための，全 Bridge 共通の必須機構は規定していない．

Controller は，取得した観測値を過去の履歴や送信 Command の期待値と比較することで，構造制約違反や期待状態との不一致を検知できる．一方，非 Matter 下位機器を対象とし，単一 Controller が Bridge から Read，Subscribe および Invoke で得る情報だけを用いるという本研究の前提範囲では，これらの情報から P_I，P_S，P_C の正当性を Bridge とは独立に確認する仕組みは規定されていない．

ConfigurationVersion など構成変更の判定を Bridge に委ねる属性を利用した場合であっても，Bridge から独立した情報源が加わるわけではない．また，観測を繰り返しても，Bridge が一貫した情報を返す場合には，下位機器との対応を裏付ける独立した情報は得られない．

4. RQ2：不整合の検知可能性の評価方法

本節では，具体的な不整合を Controller が取得する情報から検知できるかを評価する．RQ1 が仕様上の検証機構を対象としたのに対し，全 Bridge 共通の必須機構が規定されていなくても，不整合の内容によっては公開構造や報告状態に差や矛盾が現れるため，実験的評価による検証を行う．

第一に，内部構造を制御可能な SDK サンプルを用い，P_I, P_S, P_C に対応する不整合を導入した版と，対応する対照版との間で Controller 観測を比較する．第二に，市販の Matter Bridge 製品を用いて，P_I を満たさない不整合な登録を製品が受理するか，またその不整合が Controller 観測，利用者向けアプリの表示，および実機の動作へどのように伝わるかを調べる．

4.1 SDK サンプルを対象とした評価

評価には，公式 Matter SDK の Bridge サンプル（bridge-app）と Controller ツール（chip-tool）を用いた [9]．仕様分析には Matter 1.6 を用いたが，実験時点で Matter 1.6 に対応するリリースが公開されていなかったため，最新リリースである v1.5.1.0 を使用した．なお，対象とした Attribute，Command および Invoke 応答の定義について，Matter 1.5.1 と 1.6 の間に本評価へ影響する変更がないことを確認した [1], [2], [10], [11]．

bridge-app では，Endpoint 1（EP1）が Aggregator であり，Light1 は Endpoint 3（EP3）の On/Off Light として公開される．評価では，Light1 とは別に，下位機器を模擬する二つの DeviceOnOff オブジェクト（BackendLight1，BackendLight2）を追加した．本研究では，これらを模擬下位機器と呼ぶ．模擬下位機器は Endpoint として公開されず，実機や下位プロトコル通信を伴わない．EP3 の状態反映元とコマンド適用先を個別に指定する実験機構を，すべての対照版および改変版に共通して追加した．

検証対象ごとに，次の六つの不整合ケースを設定した．P_I では，対応する模擬下位機器を持たない Endpoint を追加する I1 と，EP3 の状態反映元とコマンド適用先を変更する I2 を設けた．P_S では，模擬下位機器の Reachable の変化を Matter 側へ反映しない S1 と，同じく OnOff の変化を反映しない S2 を設けた．P_C では，On/Off 要求値の変換を反転する C1 と，EP3 のコマンド適用先だけを別の模擬下位機器へ変更する C2 を設けた．各ケースでは，これらの箇所だけを追加で変更し，当該の改変を加えていない対照版を対で設けた．I1 と I2 は共通の対照版を用い，S2 では状態変化を与えない対照版に加え，同じ状態変化が反映されることを確かめる対照版も用いた．対照版は計 6 系列である．

各実行では Bridge と chip-tool の保存領域を分け，Device Attestation の検証を省略するオプションを指定して新規に Commissioning を行った．この省略はテスト用証明書を用いるためであり，対象とする不整合は Commissioning 後の内部処理に起因するため結果に影響しない．C1 および C2 では，Commissioning 後に両方の模擬下位機器を直接 ON にする刺激を与えて事前状態を整えた．Controller の On コマンドを用いないのは，C1 では要求値が反転し，C2 では別の模擬下位機器へ適用されるためである．

Controller の Read は SDK の既存経路を通じて Light1 を読み出し，模擬下位機器を直接読み出さない．観測項目はケースに応じて定めた．I1 および I2 では，PartsList, DeviceTypeList, ServerList, UniqueID, 事前の Reachable と OnOff，On と Off の Invoke 応答，および事後の OnOff の 9 項目を比較した．S1 では構成情報 4 項目と Reachable および OnOff の事前・事後値を，S2 では Reachable と OnOff の事前・事後値を比較した．

© 2026 Information Processing Society of Japan

表 1　SDK サンプルにおける Controller 観測の比較結果
Table 1　Comparison of Controller observations in the SDK sample.

検証対象	ケース	比較内容	主な観測値（対照実行／当該条件）	観測差
P_I	I1	対応する模擬下位機器なしの Endpoint 登録	`PartsList`：10 件／ 11 件	あり
P_I	I2	状態反映元とコマンド適用先	全 9 項目が一致	なし
P_S	S1	到達性の反映	`Reachable`：FALSE ／ TRUE	あり
P_S	S2	状態の反映	`OnOff`：FALSE ／ FALSE，`ΔDataVersion`：0 ／ 0	なし
P_C	C1	要求値の変換	`OnOff`：FALSE ／ TRUE	あり
P_C	C2	コマンド適用先	`Off` の Invoke 応答一致，`OnOff`：FALSE ／ FALSE	なし

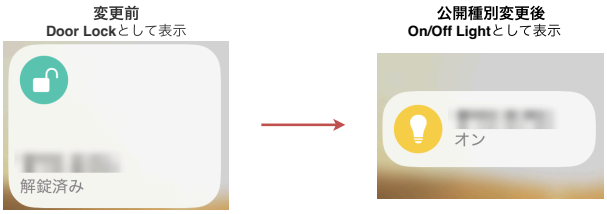


図 3　公開種別の変更に伴う利用者向けアプリ表示の変化（製品名は匿名化）
Fig. 3　Change in the user-facing application after modifying the exposed device type (product name anonymized).

C1 および C2 では，構成情報 4 項目，事前の `Reachable` と `OnOff`，`Off` の Invoke 応答，事後の `OnOff` を比較した．`DataVersion` は，同一 Cluster の事前・事後値からの増分を補助指標とした．増分の絶対値は実装に依存するため，対照版と当該条件で同一の実装を用い，一致・不一致のみを比較した．

4.2　市販 Bridge 製品を対象とした評価

本評価では，市販の Matter Bridge 製品 1 台を用い，実機種別と Matter 上の公開種別が一致しない登録を受理するか，その不整合が Controller 観測，利用者向けアプリの表示および実機動作へどのように伝わるかを調べた．本評価は特定の製品や実装の欠陥を指摘することを目的としないため，製品名は匿名化する．また，1 製品のみの評価であるため，得られた結果は対象製品における挙動のケーススタディとして記述する．

対象製品の Web 管理画面が認証済みアカウントから送信する登録要求には，実機種別と Matter 上の公開種別が別の値として含まれる．本実験では，登録要求に含まれる公開種別の値のみを書き換えて送信し，スマートロックを Door Lock から On/Off Light へ変更した．

Controller 観測として，`chip-tool` で取得した `DeviceTypeList`，`ServerList`，`ProductName`，Invoke 応答および事前・事後の `OnOff` を記録した．利用者向けアプリの表示と実機動作を記録した．

5.　RQ2：評価結果

5.1　SDK サンプルの結果

Controller 観測の比較結果と S2 の状態反映の確認結果を**表 1** に示す．

I1 では，EP1 の `PartsList` が対照実行の 10 件に対して 11 件となった．既知の基準構成があれば，Controller は Endpoint の増加を `PartsList` の差として検知できる．一方，`PartsList` だけでは，追加された Endpoint に対応する模擬下位機器が存在するかを特定できない．

I2 では，上記 9 項目のすべてが対照実行と一致した．EP3 の状態反映元とコマンド適用先を `BackendLight1` から `BackendLight2` へ変更しても，その差は Controller 観測に現れなかった．EP3 への操作要求は，対照版とは異なる模擬下位機器に適用された．

S1 では，両方の実行で `BackendLight1` の `Reachable` を FALSE へ変化させた結果，`Reachable` は対照実行で FALSE，S1 で TRUE となった．BDBI Cluster の `DataVersion` の増分も異なった．対照実行との比較では，これらの差が Controller 観測に現れた．ただし，S1 の `Reachable=TRUE` が §9.13.5.2 の偽陽性にあたるかは，模擬下位機器が到達不可であることを別に確認して初めて判定できる．

S2 では，模擬下位機器の状態を変化させたうえで反映を抑止した結果，`OnOff`，`Reachable` および `DataVersion` の増分が，状態変化を与えない対照実行と一致した．この一致が状態変更操作の失敗によるものでないことを確かめるため，改変を加えていない対照版に同じ変化を与えたところ，`OnOff` は TRUE となり `DataVersion` の増分も生じた．状態変更操作自体は有効であり，模擬下位機器の状態が変化しても報告されなければ Controller は検知できない．

C1 では，あらかじめ `OnOff=TRUE` とした状態で `Off` を送信した結果，コマンド後の `OnOff` が対照実行で FALSE，C1 で TRUE となった．`OnOff` の `DataVersion` の増分も異なった．Controller は，コマンドの意味から定まる期待値と再読出し値との不一致を検知できる．この観測だけでは，原因を Bridge 内部の要求値変換に特定できない．

© 2026 Information Processing Society of Japan

C2 では，C1 と同じ手順により，`Off` の Invoke 応答とコマンド後の `OnOff` はいずれも対照実行と一致した．一方，`Off` は対照版とは異なる模擬下位機器に適用された．これは下位プロトコルへの誤送信ではなく，コマンド適用先の選択が変わった結果である．Invoke 応答と再読出し値は，コマンド適用先を示さない．

5.2 市販 Bridge 製品の結果

対象製品は登録要求を受理し，変更後の Device Type で Endpoint を公開した．スマートロックでは，Node 内の Door Lock が 1 件から 0 件へ減り，On/Off Light が既存の 2 件から 3 件へ増えた．対象 Endpoint の Door Lock Cluster は On/Off Cluster に置き換わったが，BDBI の `ProductName` は変更前と同じであった．利用者向けアプリの表示も，**図 3** に示すように錠から照明へ変化した．

この構成で `Off` を送信すると `StatusIB.Status=0x00` が返り，スマートロックが解錠位置から施錠位置へ動いた．別の実行で `On` を送信した場合も同じ成功ステータスが返り，施錠位置から解錠位置へ動いた．アプリ上の消灯・点灯操作も，それぞれ施錠・解錠に対応した．

5.3 RQ2：不整合の検知可能性への回答

RQ2 に対し，本評価は，不整合の内容によって公開構造や報告状態の差として現れる場合がある一方，Controller 観測に差が現れず検知できない場合が存在することを示した．

I1，S1，C1 では，それぞれ構成，`Reachable`，コマンド後状態に観測差が現れたが，不整合と判定するには，既知の構成，実際の到達性または送信 Command から定まる値との比較を要する．一方，I2，S2，C2 では，EP3 に対応付ける模擬下位機器，報告状態と模擬下位状態の関係，またはコマンド適用先を変更しても観測値が対照実行と一致し，不整合は Controller 観測に現れなかった．

市販製品の評価では，下位機器と異なる公開種別が受理され，アプリ上でも照明として扱われた．これは，Controller 観測だけでは公開種別と実機種別の一致を確認できないことを示している．この結果はすべての不整合が検知不能であることを意味しないが，SDK サンプルにおける局所的な実装変更や，市販製品における登録要求中の公開種別の書換えだけで，Controller 観測から不整合を判別できない点に実用上のリスクがある．

6. 考察

仕様分析，SDK サンプルの評価，市販製品の評価のいずれでも，Controller が Bridge の提示情報を前提として機器を認識するという共通の構造的限界が確認された．この構造は，下位プロトコルや下位機器の差異を Bridge 内部へ隠蔽することで相互運用性と導入の容易さを実現する．その反面，第 2.3 節の観測範囲では，機器の対応関係，状態の反映元およびコマンドの転送先が正当かどうかの判断は，Bridge の提示情報に全面的に依存する．

Controller 観測に差が現れなかった不整合は，いずれも Bridge 内部の三つの処理，すなわち P_I の対応関係の維持，P_S の状態の反映経路，P_C のコマンド変換と転送先の決定が変更されたことに起因する．したがって検知や防止の検討では，これら内部処理の不当な変更を防ぐか，その正常性を検証する仕組みが有効な方向となる．

ただし，この対策を下位機器側に求めることは困難である．非 Matter 下位機器は仕様やプロトコルが多様であり，Core §9.12.10 が示すように Matter の製品識別体系の外にある．そのため，下位機器や下位プロトコル側に共通の識別機構や検証手順を導入することは現実的ではない．

そこで，改変を想定するソフトウェア層から分離されたハードウェア等を信頼の起点とし，上記の処理が製造者の意図通りに実行されたことを示す証拠を Controller へ提示する設計が候補となる．この設計には，証拠の発行主体と信頼の起点を定めること，Controller 側の対応や規格拡張を含めて共通に検証できること，および下位機器やプロトコルへの変更を要さないことの三つが求められる．

なお，このアプローチが示せる範囲は Bridge 内部の処理の完全性にとどまる．下位機器の物理的同一性，実コマンドの実行結果，取得状態の真正性や新鮮性，下位機器自体の侵害はカバーできない．

6.1 本研究の課題

仕様分析は，非 Matter 下位機器を扱う Bridge に共通して規定され，第 2.3 節の観測を用いて単一 Controller が利用できる検証機構を対象とした．製造者固有機能，複数 Controller の比較，外部情報源，物理観測および下位プロトコル観測を用いる方式は対象外である．

SDK サンプルの評価では模擬下位機器を用い，下位プロトコル通信は行っていない．そのため，S1 では実機の切断を，S2 では状態を反映しない時間の長さを，C2 では下位プロトコル上の転送先を評価していない．

市販製品の評価は 1 製品のケーススタディであり，結果を他製品へ一般化することはできない．また，認証済みの所有者アカウントから送信される登録要求を対象としており，未認証者による侵入や権限取得を示すものではない．

両評価の対象は選定した Device Type，Attribute および Command に限られる．今後は反復回数と対象製品を増やし，ほかの Cluster や Event を含めて評価範囲を拡張する必要がある．

7. 関連研究

Matter の信頼および管理機構について，Shashwat らは，Commissioning 時に Controller が Device Attestation

© 2026 Information Processing Society of Japan

を検証する一方，機器側には Controller の提供元を同様に検証する仕組みがない非対称性を分析した [5]．Wang らは，Controller，Fabric およびアクセス制御に関する利用者向けの表示・更新機能を市販実装と自動検査により評価し，Fabric 情報の非表示などによって利用者が制御主体を正確に把握できない問題を示した [6]．これらの研究は Matter 機器の Commissioning や Fabric 内を対象としており，Bridge と非 Matter 下位機器との対応は扱っていない．

Bridge に関して，Loos は Matter 1.0 の仕様を分析し，Bridge の侵害による機器の偽装や，保護されない下位通信のリスクを論じた [7]．Pham らは Matter と ECHONET Lite を相互接続する Bridge を構築し，その相互運用性を評価した [8]．これらに対し，本研究は Matter 1.6 の Bridge に対する必須規定を対象として，機器構成，状態の取得元およびコマンドの転送先を，Controller 観測から検証できるかを分析した．

他のスマートホーム環境について，Xu らは，IoT API 経由では設定できるものの利用者からは管理も観測もできない属性を発見する手法を示し，実証した [12]．Lin らの CP-IoT は，複数プラットフォームから抽出した自動化ルールと通信トラフィックの特徴を照合し，規則実行の異常を検出する [13]．これらの研究が IoT API や通信トラフィックなど Controller 観測外の情報を用いるのに対し，本研究は Controller 観測だけから検証できる範囲を明らかにした．

8. おわりに

本研究では，Matter 1.6 の Bridge 規定を分析し，Controller が Read，Subscribe および Invoke で得る情報だけを用いて，Bridge が公開する Endpoint が下位機器と正しく対応し，報告状態が当該機器から取得され，コマンドが適切に転送されたことを確認する共通機構は規定されていないことを明らかにした．

SDK サンプルの評価では，I1，S1，C1 で Controller が取得する情報に差が現れた一方，I2，S2，C2 では不整合を導入しても対照実行と同じ情報が得られた．したがって，少なくともこれらのケースでは，Controller は取得した情報から Bridge 内部の不整合を判別できない．また，市販 Bridge 製品では，スマートロックを照明として登録でき，アプリの消灯操作によって実機が施錠位置へ動いた．

本研究の分析と評価から，保護対象の候補は，Endpoint と下位機器の対応を管理する構成情報，状態を Matter 側へ反映する処理，コマンドの転送先を決定する処理と整理できた．今後は，これらを第三者による不正な改変から保護し，その完全性を Controller が検証できる仕組みを設計し，実装・評価する．

SDK サンプルのパッチ，実験スクリプトおよび全 12 系列のログと観測値は，`https://github.com/ta-061/matter-bridge-pocs-public` で公開する．

謝辞

本研究は JSPS 科研費 JP25K21182 の助成を受けたものである．

参考文献

[1] Connectivity Standards Alliance. Matter Specification, Version 1.6. Document 23-27349-011 (Core), 2026.

[2] Connectivity Standards Alliance. Matter Application Cluster Specification, Version 1.6. Document 23-27350-010, 2026.

[3] Connectivity Standards Alliance. Matter Device Library Specification, Version 1.6. Document 23-27351-010, 2026.

[4] Connectivity Standards Alliance. Matter Arrives Bringing A More Interoperable, Simple And Secure Internet Of Things to Life. `https://csa-iot.org/newsroom/matter-arrives/`, October 2022. Accessed August 14, 2026.

[5] Kumar Shashwat, Francis Hahn, Xinming Ou, and Anoop Singhal. Security analysis of trust on the controller in the Matter protocol specification. In *2023 IEEE Conference on Communications and Network Security (CNS)*, pages 1–6. IEEE, October 2023.

[6] Haoqiang Wang, Yiwei Fang, Yichen Liu, Ze Jin, Emma Delph, Xiaojiang Du, Qixu Liu, and Luyi Xing. Hidden and lost control: on security design risks in IoT user-facing Matter controller. In *Proceedings 2025 Network and Distributed System Security Symposium*, NDSS 2025. Internet Society, 2025.

[7] Melissa Loos. Security analysis of the Matter protocol. Master's thesis, Ulm University, Ulm, Germany, 2023.

[8] Van Cu Pham, Toan Nguyen-Mau, Marios Sioutis, and Yasuo Tan. Matter and ECHONET Lite: Similarities, differences, and a bridge solution for interoperability. *Internet of Things*, 27:101265, 2024.

[9] Project CHIP Authors. connectedhomeip. GitHub repository, `https://github.com/project-chip/connectedhomeip/`, 2026. Version v1.5.1.0, commit abcc720b48c5e59c0edcfe65c516f76ca9448aa3; accessed August 19, 2026.

[10] Connectivity Standards Alliance. Matter Specification, Version 1.5.1. Document 23-27349-010 (Core), 2026.

[11] Connectivity Standards Alliance. Matter Application Cluster Specification, Version 1.5.1. Document 23-27350-009, 2026.

[12] Xuening Xu, Chenglong Fu, Xiaojiang Du, and Bo Luo. Discovering and exploiting IoT device hidden attributes: A new vulnerability in smart homes. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, CCS '25, pages 1649–1663. Association for Computing Machinery, 2025.

[13] Hai Lin, Chenglong Li, Jiahai Yang, Zhiliang Wang, Linna Fan, and Chenxin Duan. CP-IoT: A cross-platform monitoring system for smart home. In *Proceedings 2024 Network and Distributed System Security Symposium*, NDSS 2024. Internet Society, 2024.

© 2026 Information Processing Society of Japan

研究倫理に関する記載

市販の Bridge 製品を用いた実証実験において，メーカーのブランドイメージなどに与えるネガティブインパクトを防止するため，対象製品の名称やメーカー名は本研究では明示せず，得られた挙動を他製品へ一般化しない．また，本研究で確認した不整合を発生させる具体的な手順は，悪用を防ぐため本研究では伏せる．当該手順については IPA への報告を予定している．

付　　録

A.1 Bridge が提示する情報の整理と検証対象の選定

本付録は，第 3 章の検証対象について，規範上の義務，Controller が観測できる情報とその限界，および候補の採否を確認するための台帳である．対象は Core §9.12 の非 Matter 下位機器であり，Fabric Synchronization により Matter Node を表す場合は含めない．規格原文は義務との対応に必要な短い該当句だけを示す．表中の頁は，仕様書の各頁に印字された頁番号で示す．

表 A·1　分析軸に対応する Matter 1.6 の規範文
Table A·1　Normative Matter 1.6 text corresponding to the analysis axes.

区分	規格本文の該当句	出典（仕様書頁）
抽出	“SHALL include these clusters . . . based on the conformance defined below”	Device Library §1.1.7，23[3]
P_I	“SHALL expose . . . endpoints representing . . . Bridged Devices”	Core §9.12.2，666[1]
P_S	“SHALL . . . represent those as attributes, events and/or commands”	Core §9.12.4.2，672[1]
P_C	“SHALL forward this command to the relevant Bridged Device”	Core §9.12.4.1，671[1]

A.1.1 候補要素の採否台帳

表 A·3 に，対象 Device Type から抽出した候補と採否をまとめる．M は必須，O は任意，D は deprecated，desc は要素説明により適合性が決まることを表す．BDBI の `UniqueID` は `Rev>=v4，O` と表記され，これは Otherwise Conformance により「`Rev>=v4` なら必須, それ以外は任意」を意味する（Core §7.3.11）．revision 6 では条件が成立するため必須となる．一方, 同 Cluster の `ConfigurationVersion` は `[Rev>=v6]` であり，revision 6 では任意である [1].

A.1.2 採否基準と観測範囲

候補は，非 Matter 下位機器の Bridge に適用され，単一 Controller が Matter の Read，Subscribe または Invoke で観測でき，P_I，P_S または P_C の判定に関係する場合に採用した．条件付き・任意要素は補助候補とし，Fabric Synchronization，製造者固有機能，外部情報源，物理観測および下位プロトコル観測は対象外とした．Command 後の Read は仕様が定める応答ではなく，本研究の評価手順である．

表 A·2　Controller 観測で確認できる事項と限界
Table A·2　What Controller observations can and cannot establish.

軸	主な観測	確認できる事項	単独では確認できない事項
P_I	`PartsList`, `DeviceTypeList`, `ServerList`, `UniqueID`, `ConfigurationVersion`	公開構造，識別子，申告された構成変化	Endpoint と下位機器との対応
P_S	`OnOff`, `Reachable`, Event, `DataVersion`	報告値，申告された到達性と変化	状態の取得元と下位機器での発生時点
P_C	`CommandStatusIB`, Command 後の Read	Status や期待状態との不一致	下位側への送信，送信先および変換内容

Controller は公開構造の変化，報告値の変化，Status や期待状態との矛盾を検知できる．しかし，これらはすべて Bridge を経由した観測であるため，同じ識別子・構造・報告値・応答を返す二つの実行について，下位機器との対応や処理の差を単独では判別できない．

© 2026 Information Processing Society of Japan

表 A·3 仕様から抽出した候補，要件，採否および一次仕様位置

Table A·3 Candidate elements, requirements, treatment, and primary specification locations.

候補群	Conformance・仕様上の要点	本研究での扱いと限界	一次仕様位置（仕様書頁）
認証・台帳	Device Attestation は Commissioning 対象 Node を検証．Bridge と配下は単一 Node ID・Operational Certificate を共有し，非 Matter 下位機器は Product ID を持たず DCL に登録されない	Bridge Node の認証確認に使用．個々の Endpoint と下位機器との対応は証明しない	Core §6.2.3，387；§9.12.8，674；§9.12.10，675[1]
Endpoint 構成	Root Node は Endpoint 0．Bridge は 1 個以上の Aggregator と Bridged Device を表す Endpoint 群を公開．Bridged Node と Application Device Type は同一または親子 Endpoint に配置可能	対象 Endpoint 群の抽出に使用．構成の自己申告であり物理的対応は示さない	Core §2.10，66；§9.12.2，666–670；Device §2.5，31–34，§11.2，131–134[1], [3]
Descriptor	`DeviceTypeList`，`ServerList`，`ClientList`，`PartsList` は M．`TagList` は条件付き，`EndpointUniqueID` は O	server 構成と階層を P_I へ採用．client 機能，表示上の区別，任意識別子は補助候補	Core §9.5.6，630–631[1]
Basic Information・BDBI	Basic Information の `ConfigurationVersion` は現行 revision で M．BDBI では `Reachable`，`ReachableChanged`，`UniqueID` が M，`VendorName` は desc，`NodeLabel` は O，同名の `ConfigurationVersion` は条件付き O	`UniqueID` と二つの `ConfigurationVersion` を P_I，`Reachable` を P_S へ採用．値の生成・判定主体は Bridge であり独立証拠ではない	Core §9.13.5–9.13.7，677–682；§11.1.5，798[1]
On/Off Light・On/Off	Identify，Groups，On/Off，Scenes Management server は M，Level Control server と Occupancy Sensing client は O．`OnOff` と `Off` は M で，`LT`（Lighting Feature）が M のため `On` と `Toggle` も必須	`OnOff` を P_S，`Off`/`On`/`Toggle` を P_C へ採用．物理 Identify 効果と間接・時間依存操作は実験範囲外	Device §4.1，45–46；Application §§1.5.4–1.5.7，65–70[2], [3]
版・変化・時刻	必須 Global Attribute は実装要素を自己記述．`EventList` は D．`DataVersion` は Cluster instance のメタデータ．Event timestamp は Node 上の Event record 生成時刻	適合要素と変化の補助確認に使用．下位状態の取得元，発生時刻，経過時間は証明しない	Core §7.10.3，493；§7.13，501–503；§7.14.1.2，505[1]
Interaction Model	Read・Subscribe は `ReportDataMessage` で報告．対象 3 Command は Status のみを含む `CommandStatusIB` で応答可能	状態，Invoke 応答，事後 Read を観測に採用．下位プロトコル上の送信・送信先・変換内容は含まない	Core §8.4.3，574；§8.9.4–8.9.5，604–606；Application §1.5.7，70[1], [2]
境界候補	Fabric Synchronization は Matter Node を表す．Actions は O．`KeepActive`/`ActiveChanged` は BridgedICDSupport Feature がある場合のみ	全 Bridge 共通でない機能，Bridge から独立しない情報，製造者固有機能は補助候補または対象外	Device §2.5.3.1，32，§11.2.4，132；Core §9.13.4–9.13.7，677–682，§12.6，1184–1194[1], [3]

© 2026 Information Processing Society of Japan